Palacan

TECHNICAL DOCUMENT

Verifying Palacan

How to check, yourself, that the website runs exactly the published code, and how to rerun the formal proofs and the tests.

This document is generated from the public repository, where the most recent version is always available: github.com/Sbirringa/palacan-client

Version of 9 November 2026, from commit `d1261f8`
Source: `docs/VERIFICATION.md`
© 2026 Palacan. All rights reserved.

Verifying Palacan

You do not have to trust us. This document explains how to check, yourself, that the Palacan website runs exactly the code of this repository, and how to rerun the proofs and tests of the cryptography.

---

1. The website is exactly this code

The build of the web app is **reproducible**: the same commit always produces the same files, byte for byte. The build writes the SHA-256 fingerprint of every file in `dist/palacan-impronte.json`, and that list is **signed** with the Palacan release key, a hybrid Ed25519 + ML-DSA-65 key.

With the browser extension (every time you open the site)

Install **Palacan Verifica** from `frontend/my-react-app/vite-project/estensione/` (instructions in `LEGGIMI.txt` in that folder; Chrome, Edge and Brave). Each time you open Palacan it checks:

- the signature of the fingerprint list, against the release key built into the extension;
- the fingerprint of the page and of **every** file the page loads, also later;
- the service worker, the Content-Security-Policy and the release number, which must never go backwards.

The icon is green if everything matches, red with the reason if something does not, and grey if it cannot check.

By hand (on your computer)

You need Node.js 22 and git.

```
git clone https://github.com/Sbirringa/palacan-client.git
cd palacan-client/frontend/my-react-app/vite-project
npm ci
npm run build
node scripts/verificaSito.mjs --confronta dist/palacan-impronte.json
```

`verificaSito.mjs` downloads the fingerprint list from the website, compares it with the build you have just made, and checks its signature against the public release key in `sicurezza/chiave-rilascio-pubblica.json`. Every file of the site must match.

On GitHub

Every change to this repository runs the workflow **"Build verificabile"**, which repeats the same build and publishes the fingerprint of `palacan-impronte.json` in the run summary (artifact `impronte-<commit>`, kept 90 days). That fingerprint must be the same as the one of the site.

The release key's fingerprints are: ML-DSA-65 `b17eb99909ede552…`, Ed25519 `laOmhJxypCT8…` (full key in `frontend/my-react-app/vite-project/sicurezza/chiave-rilascio-pubblica.json`).

---

2. Formal proofs

The proofs are in `formale/` and run on GitHub at every change (workflow **"Modelli formali"**). To rerun them yourself:

ProVerif (protocol models; needs opam):

```
opam install proverif
python3 formale/proverif/verifica.py
```

`verifica.py` checks that every query gives exactly the expected result: a property that is no longer proven, or an attack that the model no longer finds, makes it fail.

Property tests and mutants (need Rust and Python):

```
cd crypto/palacan-crypto
cargo test --test proprieta_pq
cd ../..
python3 formale/mutanti/prova_mutanti.py
```

Kani (Linux or macOS):

```
cargo install --locked kani-verifier
cargo kani setup
cd formale/kani
cargo kani
```

3. Cryptography module

```
cd crypto/palacan-crypto
cargo test
```

The tests include the official test vectors of ML-KEM-768 (FIPS 203) and ML-DSA-65 (FIPS 204) from independent implementations, compatibility with libolm, and the rejection of weak Curve25519 keys. The workflow **"Modulo crittografico"** runs them at every change.

4. Web app tests

```
cd frontend/my-react-app/vite-project
npm ci
npx vitest run
```

Many tests use the real cryptography (the compiled WebAssembly module), not mocks.

5. What you cannot verify from here

The server code is not public. This does not weaken the end-to-end encryption, which you can verify here, but you cannot check what the server does with the metadata it sees (`ARCHITECTURE.md`, section 3).